CPSC 250
NumPy, Plotting, and Simple Polynomial Fits

Edward Brash

1 The Big Idea

So far, we have used Python lists as containers for data.

Lists are flexible and useful, but they are not designed primarily for numerical computation.

This week, we begin using tools that are designed for scientific and data-oriented computing.

Lists are containers.

NumPy arrays are mathematical objects.

This distinction is extremely important.

2 Why NumPy?

NumPy is a Python library for efficient numerical computing.

It allows us to store and manipulate large collections of numbers.

Python List	NumPy Array
General-purpose container	Numerical data structure
Stores arbitrary objects	Stores numbers efficiently
Often requires loops	Supports vectorized operations
Flexible	Fast

3 Importing NumPy

The usual convention is:

```
import numpy as np
```

This means that we access NumPy functions using:

```
np.function_name()
```

4 Creating a NumPy Array

We can create a NumPy array from a Python list.

```
import numpy as np

values = np.array([1, 2, 3, 4, 5])

print(values)
```

This prints:

```
[1 2 3 4 5]
```

5 Arrays Behave Differently from Lists

Consider a Python list:

```
numbers = [1, 2, 3]

print(numbers * 2)
```

This produces:

```
[1, 2, 3, 1, 2, 3]
```

The list is repeated.
Now consider a NumPy array:

```
numbers = np.array([1, 2, 3])

print(numbers * 2)
```

This produces:

```
[2 4 6]
```

Each element is multiplied by 2.

6 Special Arrays

NumPy provides several useful functions for creating arrays.

6.1 Zeros and Ones

```
zeros = np.zeros(5)
ones = np.ones(5)

print(zeros)
print(ones)
```

Output:

```
[0. 0. 0. 0. 0.]
[1. 1. 1. 1. 1.]
```

6.2 Using arange

```
x = np.arange(0, 10, 1)

print(x)
```

Output:

```
[0 1 2 3 4 5 6 7 8 9]
```

The syntax is:

```
np.arange(start, stop, step)
```

7 Using linspace

For plotting and numerical work, `linspace` is often more useful.

```
x = np.linspace(0, 10, 6)

print(x)
```

Output:

```
[ 0.  2.  4.  6.  8. 10.]
```

The syntax is:

```
np.linspace(start, stop, number_of_points)
```

This creates evenly spaced values from the starting value to the ending value.

8 Vectorized Operations

Suppose we want to compute:

$$y = x^2$$

for many values of x.
With a list, we might write a loop.

```
x_values = [-2, -1, 0, 1, 2]
y_values = []

for x in x_values:
    y_values.append(x**2)

print(y_values)
```

With NumPy, we can write:

```
x = np.array([-2, -1, 0, 1, 2])

y = x**2

print(y)
```

Output:

```
[4 1 0 1 4]
```

No loop is required.

9 Vectorized Physics Example

Suppose an object moves according to:

$$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2$$

We can calculate its position at many times.

```
t = np.linspace(0, 10, 101)

x0 = 5.0
v0 = 12.0
a = -9.8

x = x0 + v0*t + 0.5*a*t**2
```

This calculation happens for every value in the array `t`.
This is the style of computation used in many scientific Python programs.

10 Useful NumPy Functions

NumPy also provides useful statistical and summary functions.

```
data = np.array([72, 75, 79, 81, 78, 74])

print(np.min(data))
print(np.max(data))
print(np.mean(data))
print(np.std(data))
```

These compute the minimum, maximum, mean, and standard deviation.

11 Plotting with Matplotlib

We usually import Matplotlib as:

```
import matplotlib.pyplot as plt
```

Suppose we have data:

```
x = np.array([1, 2, 3, 4, 5])
y = np.array([3, 5, 7, 8, 11])
```

We can make a scatter plot:

```
plt.scatter(x, y)

plt.xlabel("x")
plt.ylabel("y")

plt.show()
```

12 Adding Labels and a Title

Plots should be readable.

A plot should usually include:

- an x-axis label,
- a y-axis label,
- a title,
- a legend, if more than one curve or data set appears.

```
plt.scatter(x, y, label="Data")

plt.xlabel("x")
plt.ylabel("y")
plt.title("Example Data")

plt.legend()
plt.show()
```

13 Linear Models

Many data sets are approximately linear.

A straight line has the form:

$$y = mx + b$$

where:

- m is the slope,
- b is the intercept.

In data analysis, we often want to find the line that best fits a set of data.

14 Simple Fitting with polyfit

NumPy includes a function called `polyfit`.

For a straight-line fit:

```
m, b = np.polyfit(x, y, 1)
```

The final argument is the degree of the polynomial.

$$1 \rightarrow \text{first-degree polynomial}$$

A first-degree polynomial is a line:

$$y = mx + b$$

15 Example: Finding the Best-Fit Line

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1, 2, 3, 4, 5])
y = np.array([3, 5, 7, 8, 11])

m, b = np.polyfit(x, y, 1)

print("slope =", m)
print("intercept =", b)
```

Possible output:

```
slope = 1.9
intercept = 0.9
```

So the best-fit line is approximately:

$$y = 1.9x + 0.9$$

16 Plotting the Fit

Once we have m and b, we can compute the fitted values.

```
y_fit = m*x + b
```

Then we plot both the data and the fit.

```
plt.scatter(x, y, label="Data")

plt.plot(x, y_fit, label="Best-fit line")

plt.xlabel("x")
plt.ylabel("y")
plt.title("Data with Linear Fit")

plt.legend()
plt.show()
```

17 A Smoother Fit Line

The previous plot only draws the line at the original data points.

For a smoother-looking line, create more x-values.

```
x_fit = np.linspace(min(x), max(x), 100)

y_fit = m*x_fit + b

plt.scatter(x, y, label="Data")
plt.plot(x_fit, y_fit, label="Best-fit line")

plt.xlabel("x")
plt.ylabel("y")
plt.legend()

plt.show()
```

18 Example: Distance vs Time

Suppose we measure the position of an object at several times.

Time (s)	Distance (m)
0	1.2
1	3.1
2	5.4
3	7.0
4	9.3
5	11.1

We can store the data:

```
time = np.array([0, 1, 2, 3, 4, 5])
distance = np.array([1.2, 3.1, 5.4, 7.0, 9.3, 11.1])
```

19 Fitting the Distance Data

```
m, b = np.polyfit(time, distance, 1)

print("slope =", m)
print("intercept =", b)
```

In this example, the slope has units of:

$$\frac{\text{meters}}{\text{second}}$$

So the slope represents an estimate of the object's velocity.

20 Plotting the Distance Data

```
time_fit = np.linspace(min(time), max(time), 100)

distance_fit = m*time_fit + b

plt.scatter(time, distance, label="Measurements")
plt.plot(time_fit, distance_fit, label="Best-fit line")

plt.xlabel("Time (s)")
plt.ylabel("Distance (m)")
plt.title("Distance vs Time")

plt.legend()
plt.show()
```

21 Higher-Degree Polynomial Fits

The same function can also fit higher-degree polynomials.

For example:

```
coefficients = np.polyfit(x, y, 2)
```

This fits a quadratic function:

$$y = ax^2 + bx + c$$

However, today we will mostly focus on first-degree fits.

22 A Warning About Polynomial Fits

Just because we can fit a polynomial does not mean we should.

High-degree polynomials can wiggle in strange ways and may not represent the underlying situation.

For this course, our main goal is simple:

Use a fit to summarize a trend in the data.

23 Practice

Write a program that does the following:

1. Creates NumPy arrays for time and distance.
2. Makes a scatter plot of the data.
3. Uses `np.polyfit()` to find the best-fit line.
4. Prints the slope and intercept.
5. Plots the data and the best-fit line together.
6. Interprets the slope in words.

24 Summary

Today we learned:

- NumPy arrays are mathematical objects.
- Array operations are vectorized.
- `linspace` is useful for creating smooth ranges of values.
- Matplotlib lets us visualize numerical data.
- `np.polyfit()` can find a simple best-fit line.
- The slope and intercept of a fit should be interpreted in context.